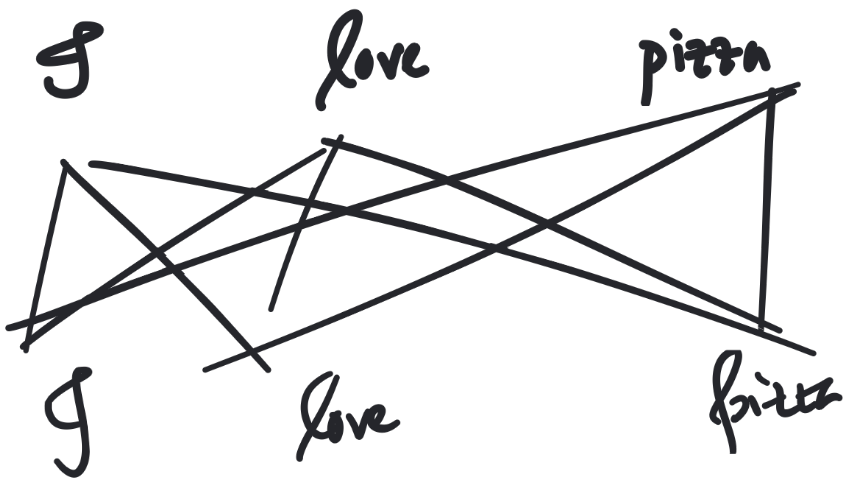
Inference Optimization



I

Q K V

What do you love?

↓

LLM

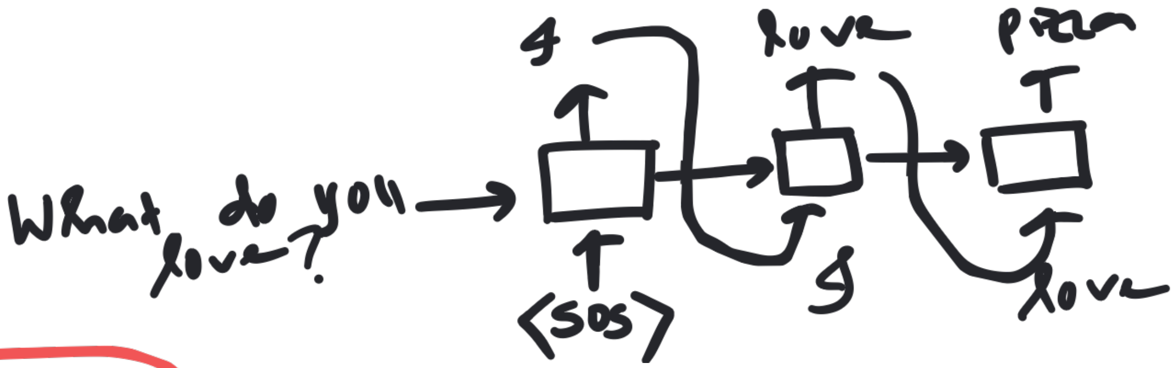
↓

I love pizza

$Q \leftarrow$ "how"

$K \leftarrow$ "info"

$V \leftarrow$ "context"



Step1 : I $\quad Q_1 \quad K_1 \quad V_1$

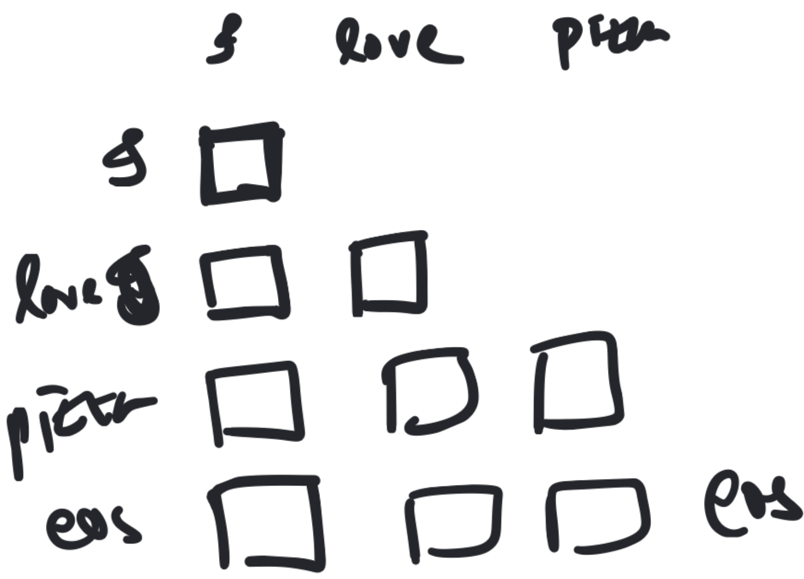
Step2 : ✓ I $\rightarrow Q_1 \quad K_1 \quad V_1$

✓ love $\rightarrow Q_2 \quad K_2 \quad V_2$

Step3 I $\rightarrow Q_1 \quad K_1 \quad V_1$

love $\rightarrow Q_2 \quad K_2 \quad V_2$

pizza $\rightarrow Q_3 \quad K_3 \quad V_3$



Step1 : token = "I"

compute → Q_1 K_1 V_1 ← I

cache - { K_1 V_1 }

reuse from cache

Step2 token love

K_1/V_1 ✗

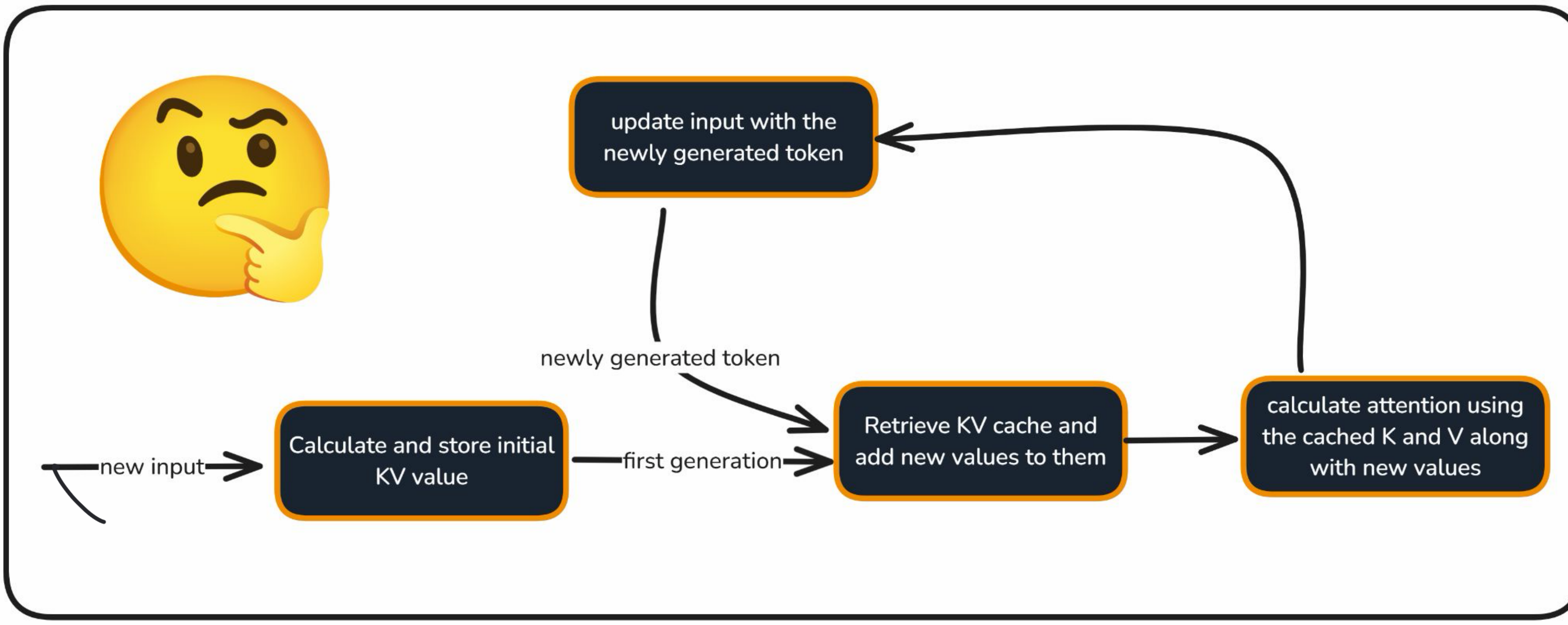
compute → Q_2 K_2 V_2

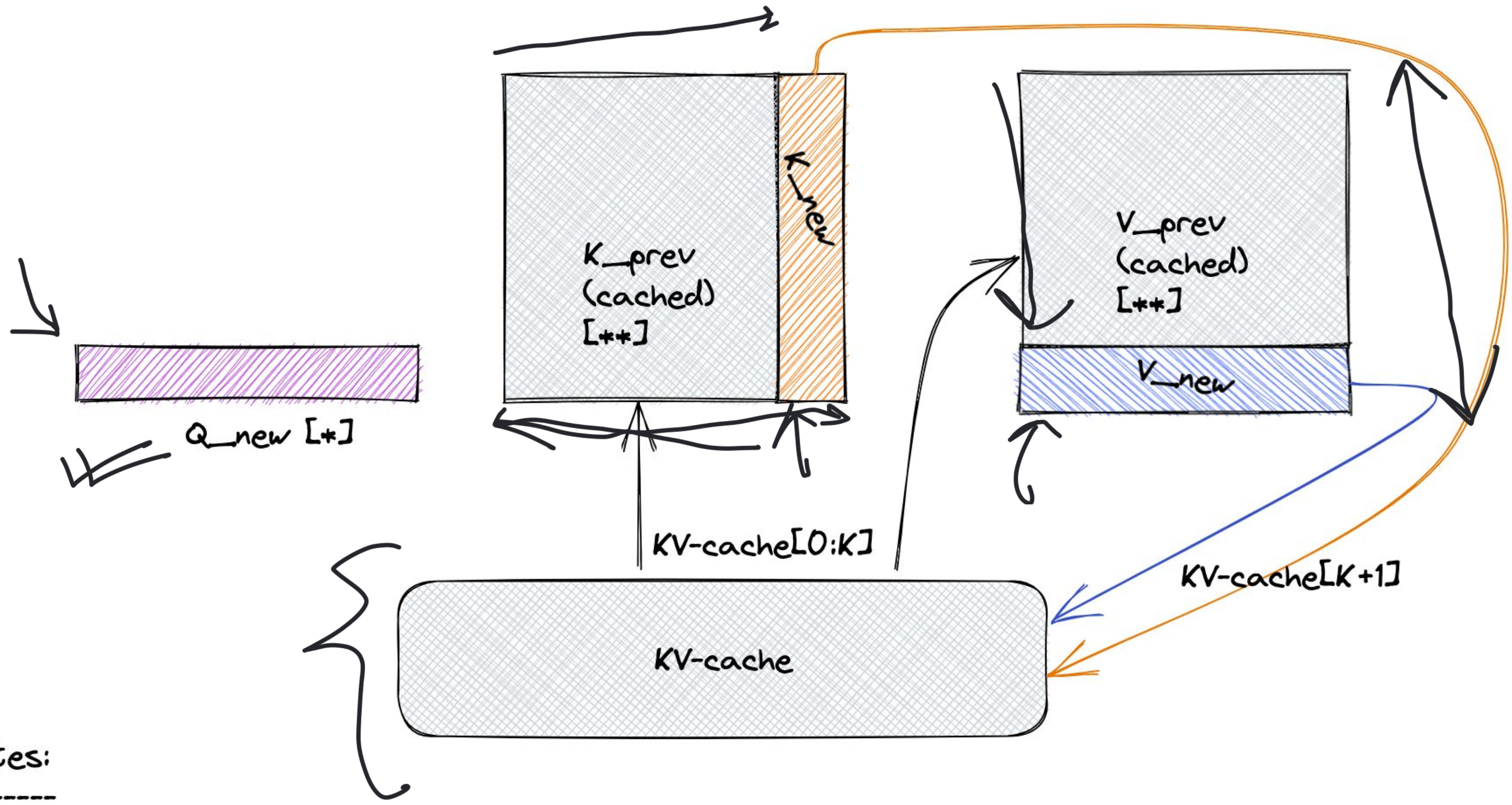
cache - { K_1 V_1 K_2 V_2 }

KV cache → memory } filled up

↳ MQA

↳ GQA


update input with the newly generated token
newly generated token
new input
Calculate and store initial KV value
first generation
Retrieve KV cache and add new values to them
calculate attention using the cached K and V along with new values

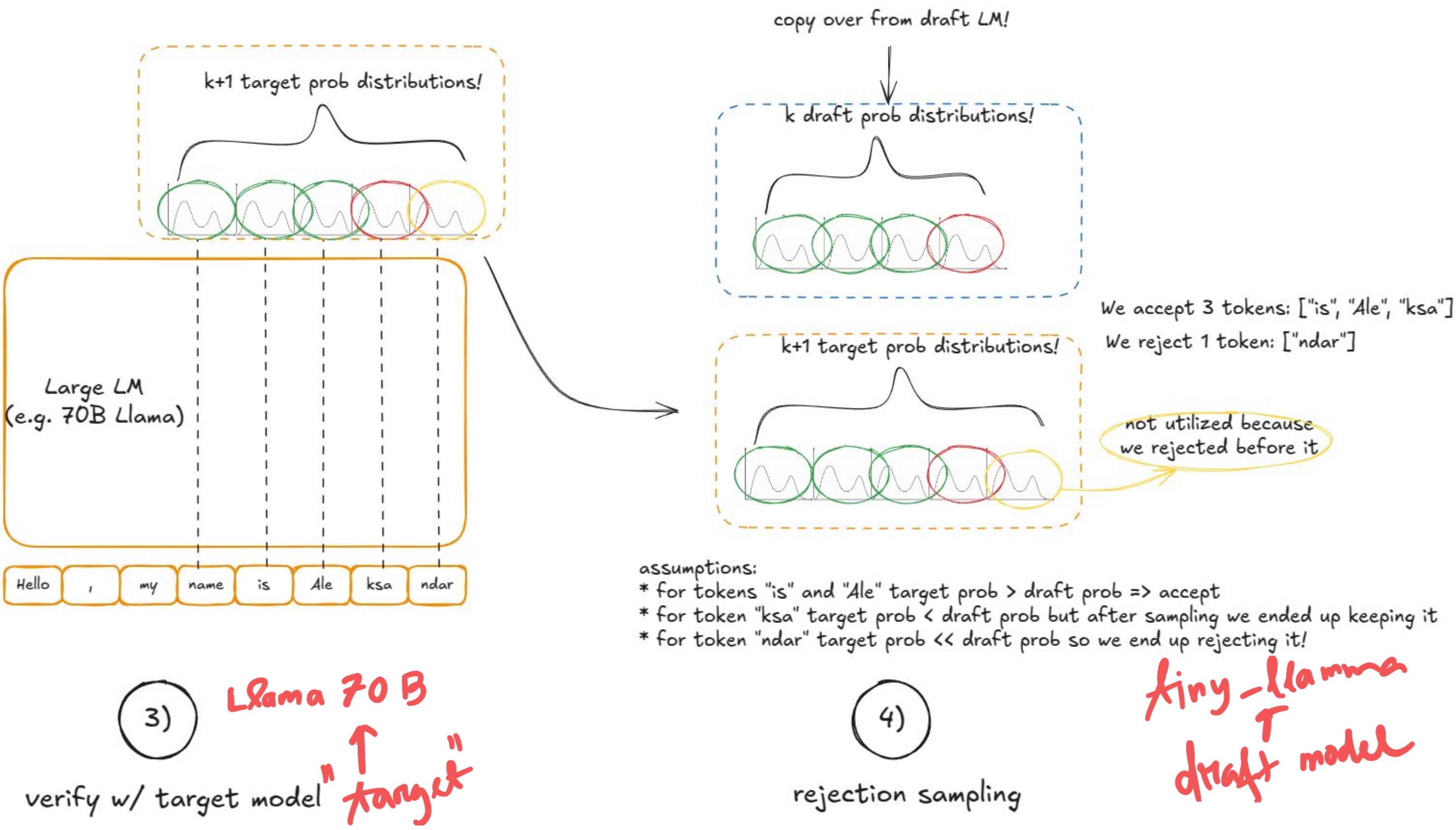


Notes:

----

* When processing token[K], we only need the K'th row of Q

** When processing token[K], we require the full K & V tensors, but we can mostly reuse the cached values
(This enables skipping the computation of K & V



We end up feeding ["Hello", ",", "my", "name", "is", "Ale", "ksa"] into the drafter - and the loop continues!

Speculative Decoding

→ Draft Model (D) Tiny llama

→ Target Model (T) Llama 70B

T

Very slow for loop

HP

K =

user

(tiny llama)

prompt → Draft

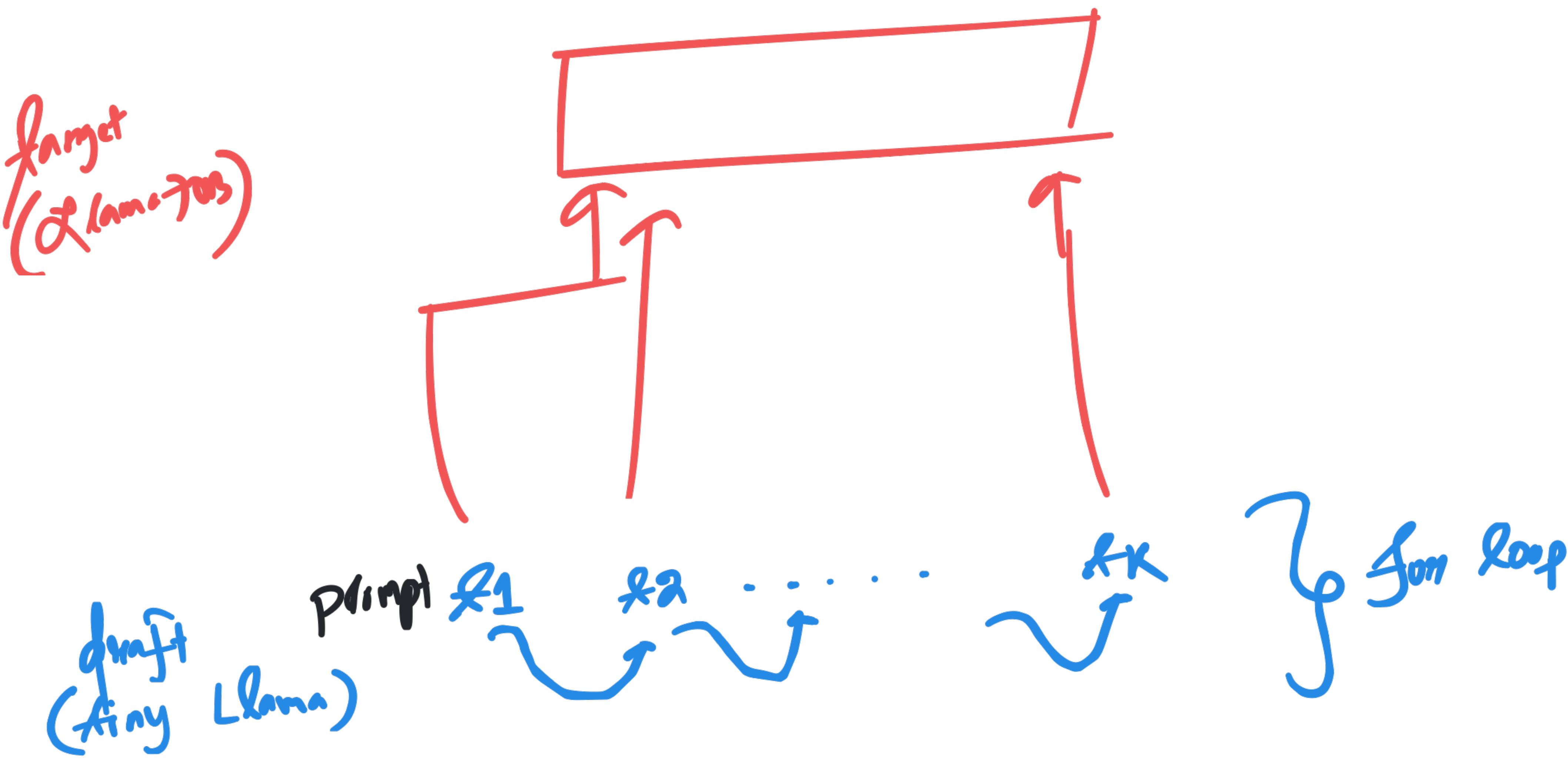
P → D → D → D . . . . D

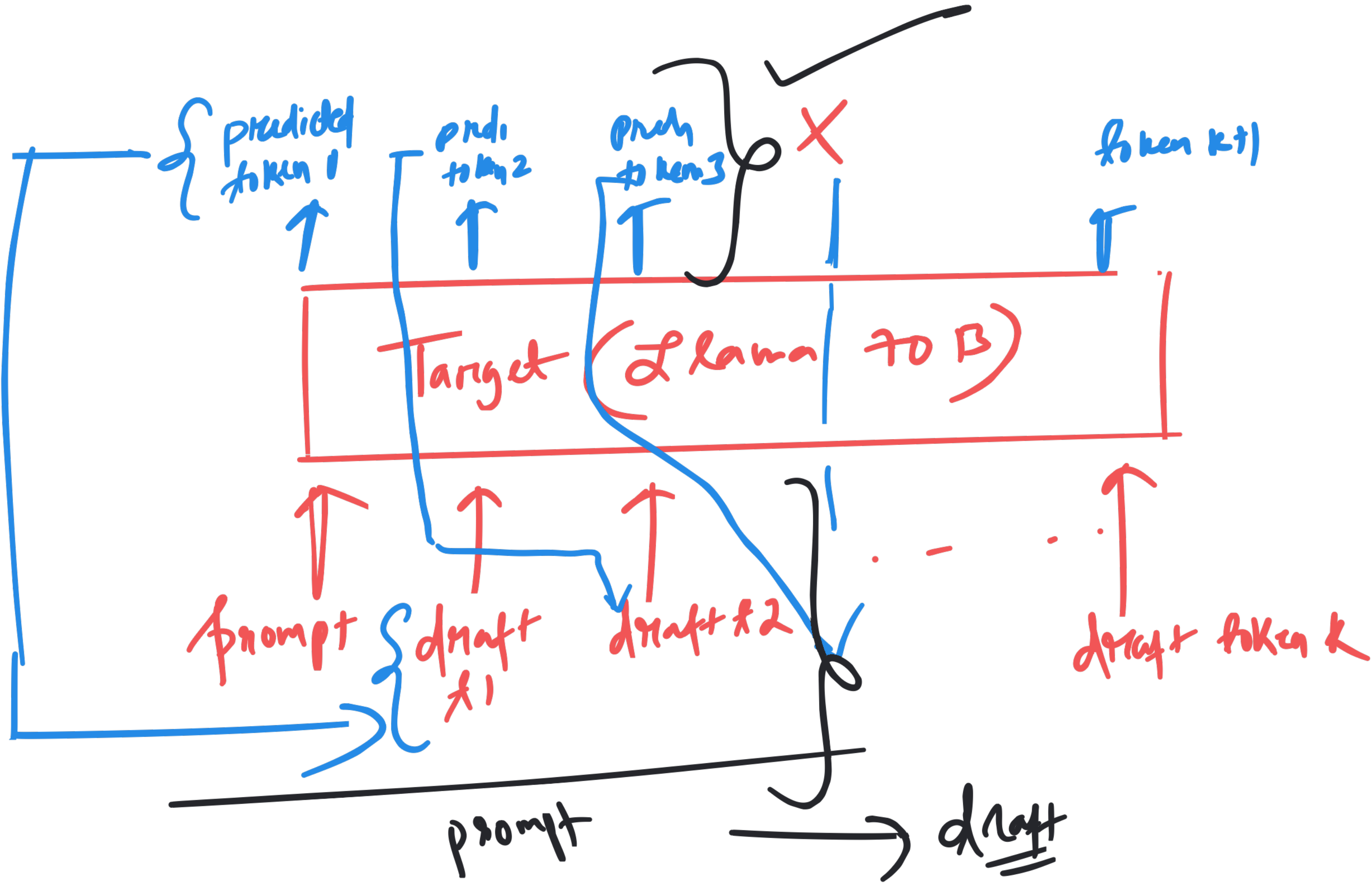
token1 token2 token k

once "k" tokens are produced

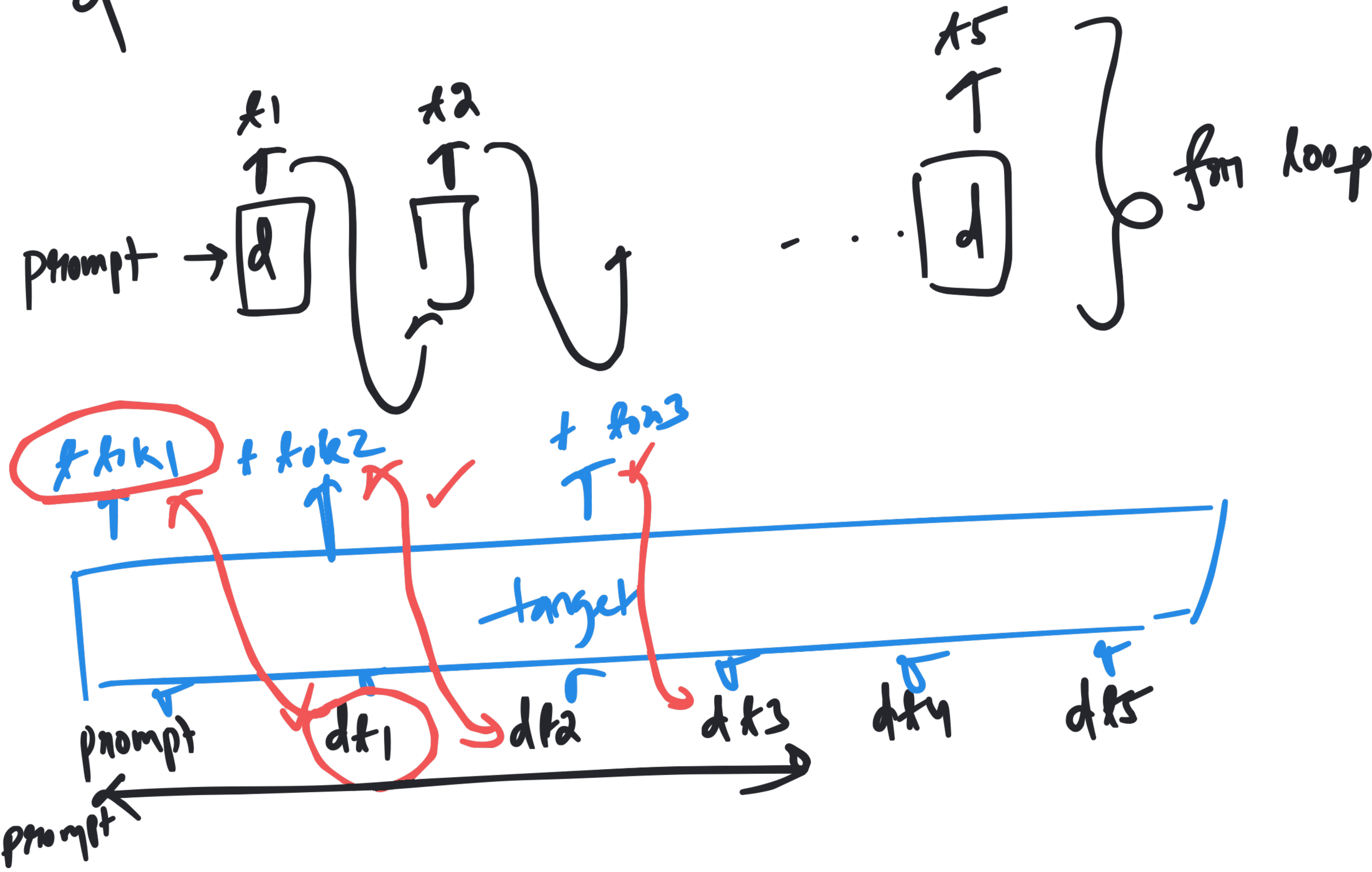
for loop or not

draft model { fast / slow }


target
(Llama-70B)
draft
(tiny Llama)
prompt
t1
t2
tk
for loop


predicted token 1
predi token 2
pred token 3
token k+1
Target (Llama 70B)
prompt
draft #1
draft #2
draft token k
prompt
draft

d


t1
t2
t5
prompt
d
d
for loop
+ tok1
+ tok2
+ tok3
target
prompt
dt1
dt2
dt3
dt4
dt5
prompt

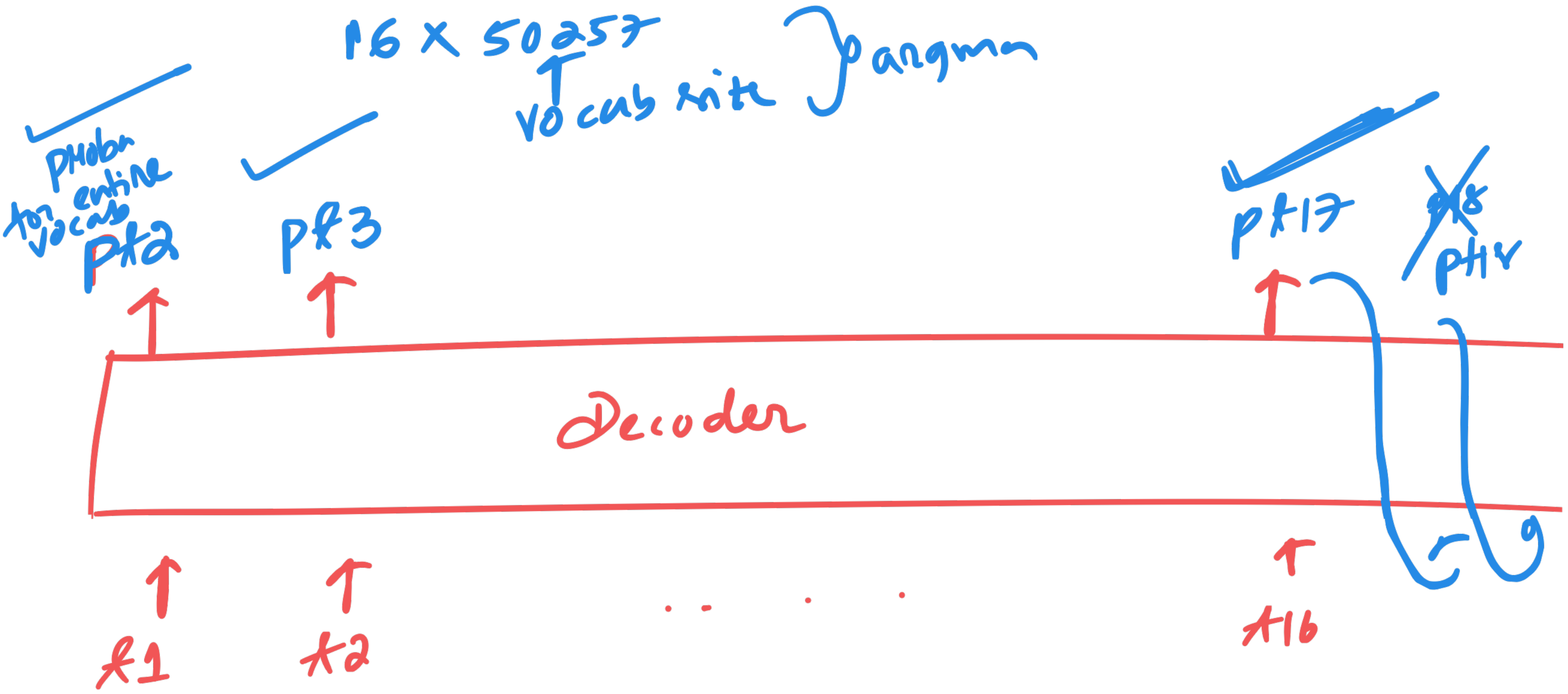
prompt $\rightarrow$ draft

prompt + k predicted tokens by draft $\rightarrow$ target

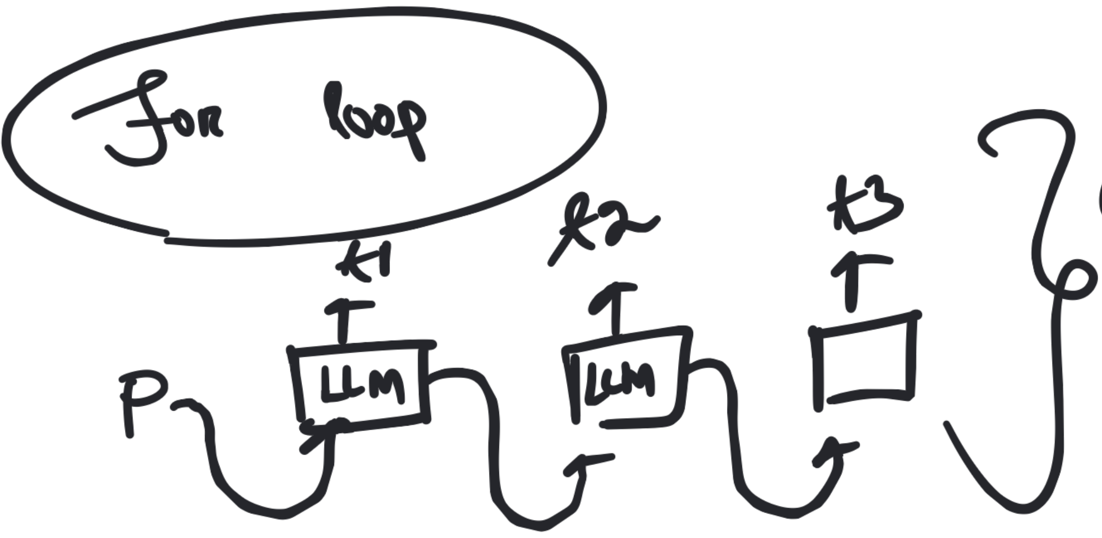
1 hr

Inference optimizat^n

Next week


16 X 50257
vocab size
argmax
prob for entire vocab
Px2
Px3
Px17
~~x18~~
Px18
Decoder
x1
x2
x16
. . . .

prompt → Decoder → 1st token + prompt → Decoder → 2nd token + 1st token + prompt → Decoder → 3rd token



Can we have the decoding in one go?

NO

Issue when the model is large